$\phi(n)$-EVALUATION ALGORITHM: A NOVEL APPROACH FOR AN EFFICIENT RETRIEVAL OF EULER'S TOTIENT OF AN RSA MODULUS

JAY MEHTA AND HITARTH RANA†

Abstract. In this paper, we propose an algorithm to efficiently retrieve the value of Euler's totient function of an RSA modulus, consequently voiding the RSA encryption. Furthermore, we show that the proposed algorithm is significantly faster and more effective when the prime factors of an RSA modulus are closer to each other. We conjecture a relation between the difference of two prime factors of the RSA modulus and the required number of steps for the algorithm.

1. Introduction

The renowned RSA cryptosystem [6], introduced in 1978, has been a center of focus for numerous researchers working in cryptology from the outset. This system revolves around a significant mathematical problem known as the integer factorization problem. As an asymmetric key cryptosystem, it uses a public encryption key to encrypt the messages sent to the receiver, and a private key known only to the receiver to decrypt the ciphertext.

In the RSA cryptosystem, initially, two secret large primes, say p and q are chosen, and their product $n = pq$ is released publicly. Further, the recipient chooses an integer e such that it is relatively prime with $\phi(n)$. This integer is also made public for the purpose of encryption. Subsequently, another integer d is computed from e as its modular inverse modulo $\phi(n)$. Using their public information e and n, anyone can send a message to the recipient. However, only the receiver can retrieve these messages using their confidential key d. The detailed RSA cryptosystem is described in Section 2.

It is well known that compromising an RSA encryption, i.e., finding the secret decryption key, amounts to solving the integer factorization problem. In the historical context of number theory and cryptography, there have been various endeavors aimed at solving this problem. One such method is given by John Pollard [5], which is effective when the

Date: February 10, 2026.

2020 *Mathematics Subject Classification.* 11T71, 94A60.

Key words and phrases. RSA, cryptanalysis, Euler's totient, factorization, prime gap, bounds for $\phi(n)$.

†Corresponding Author.

prime factors of $p-1$ or $q-1$ are relatively smaller. An analogue of Pollard's algorithm in elliptic curve cryptography is known as Lenstra's elliptic curve factorization (ECF) algorithm [3]. In this paper, we introduce a novel approach to attack RSA by proposing an algorithm, called the $\phi(n)$-*evaluation algorithm* (see Algorithm 1 in Section 3.2), which is effective when the prime factors p and q of the RSA modulus n are closer to each other, preferably of the same bit-size.

The historical study has witnessed that attacks on RSA are not limited to the ones mentioned above. Since its inception, the list of attacks against it has been constantly elongating. Some of the members in this list include the factorization of RSA modulus by the Number Field Sieve (NFS) technique, the abuse of encryption or decryption exponents of small bit size, known least or most significant bits of the private key, known partial factorization, Chinese remainder theorem attack, etc. A comprehensive survey of all these attacks was conducted in [1], which was later updated in [4]. All these attacks are also studied for a multi-prime version of RSA in [2].

In order to recover the secret decryption key as the modular inverse public encryption key, modulo $\phi(n)$, one would therefore require the knowledge of Euler's totient $\phi(n)$. Expanding on this rationale, in this paper, we propose the $\phi(n)$-evaluation algorithm mentioned above to efficiently retrieve the value of $\phi(n) = (p-1)(q-1)$, for $n = pq$, without the knowledge of the prime factors p and q in Section 3. We begin by obtaining an upper bound for $\phi(n)$ to initialize our algorithm (see Theorem 1 in Section 3.1). Thereafter, we obtain a condition to terminate the algorithm. Building on these two foundations, we introduce our algorithm (see Section 3.2). Commencing from the upper bound, at each iteration of the algorithm, we assess whether the termination condition is met before commencing the next iteration. When the termination condition is met, the algorithm yields the value of $\phi(n)$. Ultimately, we discuss the computational efficiency of our algorithm by obtaining a generous lower bound for $\phi(n)$ in Section 3.3. Further, in Section 4, we analyze the time-efficiency of the proposed $\phi(n)$-evaluation algorithm and also give a better lower bound in terms of the difference between the two, assumably unknown, prime factors of the RSA modulus n. We also conjecture a specific relation between the difference of two prime factors of the RSA modulus n and the difference of $\phi(n)$ and its upper bound obtained earlier (see Conjecture 1), which gives a better approximation of the time-efficiency of the proposed algorithm.

We conclude the paper by appending a conclusion section (see Section 5) followed by Section 6 describing further scopes of research along this line.

As a prerequisite of this paper, we recall the RSA cryptosystem, distributed in three different phases, in the following section.

2. Preliminaries

The RSA Cryptosystem. The implementation of the RSA cryptosystem is carried out in three fundamental steps, namely, key generation, encryption, and decryption.

To establish the RSA cryptosystem, the recipient, of the messages to be communicated, chooses two large prime numbers, p and q. The primes remain classified while their product $n = pq$ is published.

(1) Key generation: The recipient selects an integer e modulo $\phi(n)$ such that it is relatively prime to $\phi(n)$. Subsequently, another integer d is computed as its modular inverse modulo $\phi(n)$. i.e.,

$$de \equiv 1 \pmod{\phi(n)}.$$

The integer e is published alongside n for encryption purposes, while d is kept undisclosed.

(2) Encryption: The sender of the message is able to encrypt their message m using the public knowledge of e and n, as

$$c \equiv m^e \pmod{n}.$$

The resultant ciphertext c is sent via an insecure communication channel instead of the original message m. The integer e, being public and utilized in encryption, is often referred to as the public key or the encryption exponent.

(3) Decryption: The recipient receives the ciphertext c and proceeds to decipher it using their confidential knowledge of d. They calculate

$$c^d \equiv m^{ed} \equiv m \pmod{N},$$

and derive the plaintext m. Since the integer d is exclusive to the recipient and is employed in decryption, it is also known as the private key or the decryption exponent. The integer n, which serves as a common modulus in both encryption and decryption operations, is hence termed the RSA modulus.

Observing the RSA cryptosystem, it can be noticed that an adversary requires the value of the private key d to decrypt the ciphertext c and breach encryption. The only insight we have on the value of d is the congruential relation $e \cdot d \equiv 1 \pmod{\phi(n)}$. To ascertain the value of d from this congruence, we require the value of $\phi(n)$. For the RSA modulus $n = pq$, the value of $\phi(n)$ can be given by $\phi(n) = (p-1)(q-1)$. In despair of the intruder, the prime factors of n are confidential and recovering the value of $\phi(n)$ without them is extremely time-consuming and hence unfavorable.

In what follows, we attempt at determining this value of $\phi(n)$ in a distinctive manner. We propose an algorithm and conjecture that it is quite efficient under a certain condition.

3. Proposed Algorithm: The $\phi(n)$-evaluation algorithm

Referring again to the known formula for evaluating $\phi(n)$, we observe that $\phi(n) = (p-1)(q-1) = pq - (p+q) + 1 = n - (p+q) + 1$. Thus, it can be easily deduced that the problem of determining the value of $\phi(n)$ is as difficult as the problem of determining the value of $p+q$.

It is also evident and well-known that the computation of $\phi(n)$ is no easier than factorizing the RSA modulus n. Consequently, the primary objective becomes recovering p and q. As an approach of finding p and q, consider the quadratic equation $(t-p)(t-q) = 0$, i.e., $t^2 - (p+q)t + pq = 0$, i.e.,

$$t^2 - (p+q)t + n = 0. \tag{1}$$

Thus, once $p+q$ is known, the primes p and q can be obtained as the roots of equation (1) as follows.

$$p, q = \frac{-(p+q) \pm \sqrt{D}}{2},$$

where $D = (p+q)^2 - 4n$. Since p and q are integers, the value of $\sqrt{D}$ must be an integer. Hence, our goal is to determine a value of D, which is a perfect square.

Note that an adversary is ignorant of the value of $p+q$ in equation (1). Hence, we must investigate the potential candidates for $p+q$. In order to arrive at the precise value of $p+q$, a suitable confined range must be determined. We begin by determining an upper bound for $\phi(n)$.

3.1. An upper bound for $\phi(n)$.

Consider an RSA cryptosystem with modulus $n = pq$, where p and q are distinct concealed primes. As mentioned above, to breach the RSA encryption, we require an upper bound for $\phi(n)$ to initiate. We determine this upper bound in terms of the known value of $n = pq$, and under the assumption that the primes p and q are unknown. The following result gives a very precise upper bound for $\phi(n)$.

Theorem 1 (An upper bound for $\phi(pq)$)**.** For $n = pq$, where p and q distinct primes, an upper bound for $\phi(n)$ is given by $u = \lfloor (\sqrt{n} - 1)^2 \rfloor$.

Proof. We know that,

$$\begin{aligned} & (p-q)^2 > 0 && (\because p \neq q) \\ \Rightarrow\ & p^2 - 2pq + q^2 > 0 \\ \Rightarrow\ & (p+q)^2 > 4n && (\because n = pq) \\ \Rightarrow\ & p + q > 2\sqrt{n}. && (2) \end{aligned}$$

Now, for distinct primes p, q and $n = pq$, we have $\phi(n) = (p-1)(q-1) = pq - (p+q) + 1$.

Using the inequality in (2), we get $\phi(n) < n - 2\sqrt{n} + 1$ and so $\phi(n) < (\sqrt{n} - 1)^2$. But $\phi(n)$ being an integer, we can write

$$\phi(n) \leq \lfloor (\sqrt{n} - 1)^2 \rfloor. \tag{3}$$

□

Remark 1. Observe that p and q are distinct prime numbers. By exemption of the impractical trivial cases where either of them is 2, we can consider both p and q to be odd. Since $\phi(pq) = (p-1)(q-1)$, $\phi(pq)$ is a multiple of 4. Hence, we can further refine the upper bound obtained in Theorem 1 given by (3) as

$$u = 4 \left\lfloor \frac{(\sqrt{n} - 1)^2}{4} \right\rfloor. \tag{4}$$

Remark 2. Note that the upper bound u for $\phi(n)$ given by (4) is in fact the least upper bound for the set

$$\{\phi(n) \mid n = pq,\ p, q \in \mathcal{P},\ p \neq q\},$$

where $\mathcal{P}$ is the set of odd primes, as u is attained by the above set for many pairs of primes p and q. For instance, $p = 257, q = 283$ is one of such pairs, as $u = 72192 = \phi(n)$. Thus, the upper bound u established above is the most accurate upper bound for $\phi(n)$, and cannot be further optimized because the value of the upper bound u turns out to be the actual value of $\phi(n)$ in some cases.

3.2. **The $\phi(n)$-evaluation algorithm.** Now that we have a precise upper bound for the value of $\phi(n)$, we shall consider the candidates for $\phi(n)$ initiating and descending from u. Let us denote this candidate for $\phi(n)$ under consideration as x. Since we already established in Remark 1 that the value of $\phi(n)$ is a multiple of 4, in each iteration of the proposed algorithm, we deduct 4 from the value of x until we reach the precise value of $\phi(n)$, where the algorithm terminates. Let us denote the set of all possible candidates for $\phi(n)$ by X, which can be defined as

$$X = \{x \in \mathbb{N} \mid \phi(n) \leq x \leq u,\ 4 \mid x\}.$$

Since $p + q = n - \phi(n) + 1$, the candidate for $p + q$, denoted by y, can be calculated as $y = n - x + 1$. Likewise, let us denote the set of all possible candidates y for the value of $p + q$ as Y, i.e., $Y = \{y \in \mathbb{N} \mid y = n - x + 1,\ x \in X\}$. Evidently $n - u + 1 \leq y \leq p + q$, u being an upper bound for $\phi(n)$. For each such value of y, a candidate for $p+q$, we can calculate the value of the discriminant D of the quadratic

equation $t^2 - yt + n = 0$ and test whether it is a perfect square. We prove that (see Theorem 2), among all probable candidates y for the value of $p+q$, there is exactly one value y_0 such that the discriminant is a perfect square. In other words, there is a unique $y_0 \in Y$ for which the discriminant D_0 of equation $t^2 - y_0 t + n = 0$ is a perfect square. Moreover, this integer y_0 is the exact value of $p+q$ as proved in Theorem 2 below. Obtaining this value $\sqrt{D_0} \in \mathbb{N}$ concludes our quest for the value of $p+q$, along with the value of $\phi(n)$.

Summarizing the above procedure, to stumble upon this value of $\sqrt{D_0} \in \mathbb{N}$, we begin from the upper bound u for the value of $\phi(n)$ derived as (4) and then descend with an appropriate step counter. Upon descending from u, whatever value of x we arrive at is a plausible candidate for the value of $\phi(n)$ (equivalently $p+q$). For each value x in an iteration, we calculate $y = n - x + 1$. Our task is accomplished when this value y in some iteration, say y_0, is precisely $p+q$, which is obtained when $\sqrt{D_0} = \sqrt{y_0^2 - 4n}$ is an integer. Consequently, $\phi(n) = x_0 = n - y_0 + 1$ is also retrieved.

The algorithm that depicts the aforementioned process of obtaining $\phi(n)$ is as follows. Recall that the upper bound of $\phi(n)$ is $u = 4\left\lfloor \left(\frac{\sqrt{n}-1)^2}{4}\right)\right\rfloor$.

Algorithm 1: The $\phi(n)$-evaluation algorithm.

Input: $n = pq$, $u = 4\left\lfloor \left(\frac{\sqrt{n}-1)^2}{4}\right)\right\rfloor$

Output: x_0, the determined value of $\phi(n)$

```
let x = u, y_0 = 1 and x_0 = 1                    // setting initial values
while y_0 = 1 do
    let y = n − x + 1
    let D = y^2 − 4n
    if ⌊√D⌋ == √D then
        y_0 = y                                   // value of p + q is determined
        x_0 = x                                   // value of φ(n) is determined
    else
        x = x − 4                                 // descending the counter
```

Before proceeding further to execute the algorithm for some examples, we prove the aforementioned result which states that the value of the discriminant D of equation $t^2 - yt + n = 0$, for which $\sqrt{D}$ is an integer, is unique and the corresponding value of y is $p+q$.

Theorem 2. Let p and q be two distinct odd primes and $n = pq$. Let $A = \{\sqrt{D} \in \mathbb{R} \mid D = y^2 - 4n,\ y \in Y\} \cap \mathbb{N}$. Then the set A is singleton. Furthermore, if $A = \{D_0\}$, where $D_0 = y_0^2 - 4n$, then $y_0 = p+q$.

Proof. We know that $D = y^2 - 4n$ is the discriminant of the quadratic equation

$$t^2 - yt + n = 0. \tag{5}$$

Suppose t_1 and t_2 are two solutions of the given quadratic equation, then our equation can be rewritten as $(t - t_1)(t - t_2) = 0$, i.e.

$$t^2 - (t_1 + t_2)t + t_1 t_2 = 0.$$

Comparing this with equation (5), we get $t_1 + t_2 = y$ and $t_1 t_2 = n$. Since $n = pq$, there are only two possible integer solution for t_1 and t_2.

1) $t_1 = 1,\ t_2 = n$
2) $t_1 = p,\ t_2 = q$

Let us first consider the solutions 1 and n, then $y = t_1 + t_2 = n + 1$. However, we have $n - u + 1 \leq y \leq p + q$ and $u \neq 0$. Hence, the value of y cannot be $n + 1$.

Thus, the only possible integer solution of the above quadratic equation is $t_1 = p,\ t_2 = q$.

Now, we know that

$$t = \frac{-y + \sqrt{D}}{2}.$$

Since y is always an even integer, for the above equation to have integer roots, we must have the value of $\sqrt{D}$ be an even integer. Since we have already established that the number of solution pairs of equation (5) with integer values is exactly one, it suffices to prove that if the value of $\sqrt{D}$ is an integer, it must be even.

Now, in quadratic equation (5), the quantity $y = n - x + 1$ with $x \in X$ is an even number as $4 \mid x$ and $n = pq$ is odd.
Hence, we get

$$\begin{aligned} & 2 \mid y = n - x + 1 && (\because\ 4 \mid x,\ 2 \nmid n) \\ \Rightarrow\ & 4 \mid D = y^2 - 4n \end{aligned}$$

Hence, we can write $D = 4z$, where $z = \frac{y^2 - 4n}{4} \in \mathbb{Z}$.
Equivalently, $\sqrt{D} = 2\sqrt{z}$. In order for $\sqrt{D}$ to be an integer, z has to be a perfect square, and in that case, $\sqrt{D} = 2\sqrt{z}$ will always be an even integer.

This proves that the set A is singleton. Furthermore, if $A = \{D_0\}$, where $D_0 = y_0^2 - 4n$, then clearly $y_0 = p + q$ as argued above. □

Let us now comprehend the proposed $\phi(n)$-*evaluation algorithm* through some examples. A set of five examples for different values of n to compute $\phi(n)$ by Algorithm 1 is given in the following table.

p	q	$n = pq$	$u = 4\left\lfloor \frac{(\sqrt{n}-1)^2}{4} \right\rfloor$	$\phi(n) = (p-1)(q-1)$	$s = \frac{u-\phi(n)}{4}$
47017	59447	2795019599	2794913864	2794913136	182
56479	43711	2468753569	2468654196	2468653380	204
32801	57737	1893831337	1893744300	1893740800	875
28499	63313	1804357187	1804272232	1804265376	1714
65519	130973	8581219987	8581034716	8581023496	2805

TABLE 1. Some examples for Algorithm 1

In Table 1, s represents the number of steps required by the $\phi(n)$-evaluation algorithm to achieve the desired value of $p+q$, equivalently $\phi(n)$.

The apparent question is how many steps will Algorithm 1 require to successfully obtain the value of $\phi(n)$? In other words, we require an integer such that the algorithm will undoubtedly cease, providing the value of $\phi(n)$ before the value of our counter x reaches this integer. We call this integer a lower bound for the value of $\phi(n)$. In the succeeding section, we determine a generous lower bound for $\phi(n)$ and also account an observation that during practical implementations, Algorithm 1 terminates successfully well before hitting this lower bound.

3.3. **A lower bound for $\phi(n)$.** For $n = pq$, where p and q are distinct unknown primes, necessarily of the same bit-size, the following result gives a lower bound for the value of $\phi(n)$ in terms of n.

Theorem 3 (A lower bound for $\phi(pq)$)**.** Let p and q be two prime factors of the RSA modulus $n = pq$. If p and q are of the same bit-size, then $\phi(n) \geq \lceil n - 3\sqrt{n} + 1 \rceil$.

Proof. Since p and q are of the same bit-size, we can write

$$p < q < 2p.$$

Since $n = pq$ and $p < q$, we get

$$\begin{aligned} & p < \sqrt{n} \\ \Rightarrow\ & q < 2\sqrt{n} && (\because q < 2p) \\ \Rightarrow\ & p + q < 3\sqrt{n} \end{aligned}$$

We know that

$$\begin{aligned} & \phi(n) = n - (p+q) + 1 \\ \Rightarrow\ & \phi(n) > n - 3\sqrt{n} + 1 \\ \Rightarrow\ & \phi(n) \geq \lceil n - 3\sqrt{n} + 1 \rceil && (\because \phi(n) \in \mathbb{N}) \end{aligned}$$

Hence,

$$\phi(n) \geq \lceil n - 3\sqrt{n} + 1 \rceil \tag{6}$$

□

We again acknowledge that $\phi(n)$, the upper bound u, as well as the step-size in the algorithm is a multiple of 4. As a result, the lower bound obtained in (6) can be amended to be a multiple of 4 and denoted as our generous lower bound l given by

$$l = 4\left\lceil \frac{n - 3\sqrt{n} + 1}{4} \right\rceil. \tag{7}$$

Recall that the upper bound of the value of $\phi(n)$ is $u = 4\left\lfloor \frac{(\sqrt{n}-1)^2}{4} \right\rfloor$. Consequently, the time efficiency of our algorithm, as determined by the lower bound l, is $\mathcal{O}(\sqrt{n})$. Although, in practice, the actual value of $\phi(n)$ turns out to be closer to the upper bound u obtained in (4) as compared to the lower bound l obtained in (7). Thus, the lower bound l is very lenient.

We revisit the same set of five examples considered in Table 1 to illustrate that the difference between l and $\phi(n)$ is, in practice, much larger than the difference between u and $\phi(n)$.

$n = pq$	$l = 4\left\lceil \frac{n-3\sqrt{n}+1}{4} \right\rceil$	$u = 4\left\lfloor \frac{(\sqrt{n}-1)^2}{4} \right\rfloor$	$\phi(n) = (p-1)(q-1)$	$d_1 = \phi(n) - l$	$d_2 = u - \phi(n)$
2795019599	2794861000	2794913864	2794913136	52136	728
2468753569	2468604512	2468654196	2468653380	48868	816
1893831337	1893700784	1893744300	1893740800	40016	3500
1804357187	1804229756	1804272232	1804265376	35620	6856
8581219987	8580942084	8581034716	8581023496	81412	11220

TABLE 2. Distance of $\phi(n)$ from upper bound and lower bound

Table 2 demonstrates that the distance between the lower bound l and $\phi(n)$, denoted as d_1, is significantly greater than the distance between the upper bound u and $\phi(n)$, i.e., d_2. As we commence the algorithm from the upper bound u, we can successfully determine the value of $\phi(n)$ in steps that are significantly less than $\mathcal{O}(\sqrt{n})$.

In fact, in the following section, we show that there is a potential to refine this generous lower bound, by exhibiting a correlation between the distance of $\phi(n)$ from the upper bound u, and the difference of the primes p and q.

4. Time-efficiency analysis of Algorithm 1

In this section, we exhibit an interesting observation that the difference between the actual value of $\phi(n)$ and its upper bound u is always less than the difference between two prime factors of the RSA modulus. This does not give an actual lower bound for the Algorithm 1, but indicates that the algorithm must essentially terminate much before the lower bound is reached. The result is proved below.

Theorem 4. Let p and q be two distinct prime factors of the RSA modulus $n = pq$, then the distance between $\phi(n)$ and the upper bound

on the value of $\phi(n)$ is less than the distance between two primes. In other words, $u - \phi(n) \leq |q - p|$, where $u = 4\left\lfloor \frac{(\sqrt{n}-1)^2}{4} \right\rfloor$.

Proof. We know that the upper bound on the value of $\phi(n)$ is given by $u = 4\left\lfloor \frac{(\sqrt{n}-1)^2}{4} \right\rfloor$. Thus, $u \leq (\sqrt{n}-1)^2$. Now,

$$\begin{aligned} u - \phi(n) &\leq (\sqrt{n}-1)^2 - (n - (p+q) + 1) \\ &= n - 2\sqrt{n} + 1 - (n - (p+q) + 1) \\ &= 2\sqrt{n} - (p+q) \\ &= p - 2\sqrt{pq} + q \\ &= |\sqrt{p} - \sqrt{q}|^2 \\ &\leq |\sqrt{p} - \sqrt{q}|(\sqrt{p} + \sqrt{q}) \\ &= |q - p|. \end{aligned}$$

This completes the proof. □

Remark 3. From Theorem 4, we have

$$\phi(n) \geq u - |q - p|, \tag{8}$$

where u is the upper bound derived in (4). Under the assumption of this section, the primes p and q are of the same bit size. Consequently, $|q - p| < \sqrt{n}$. Note that for the lower bound l derived in (7), we have $l \approx u - \sqrt{n}$. Thus, the lower bound obtained in (8) is better compared to l. The lower bound $u - |q - p|$ cannot be applied, from an adversary point of view, since, in practice, the primes p and q are assumed to be unknown to an adversary. However, as mentioned above, Theorem 4 gives a significant relation between the difference of $\phi(n)$ and u with the difference of p and q, stating that the former cannot be more than the latter.

In the remainder of this section, we present some observations regarding the correlation between the proximity of primes and the proximity of $\phi(n)$ to u.

Reconsider the set of five examples taken in Table 1. We examine the correlation between the difference of primes and the distance d_2 between $\phi(n)$ and the upper bound u given in Table 3 below.

p	q	$\lvert p-q\rvert$	$n = pq$	$u = 4\left\lfloor \frac{(\sqrt{n}-1)^2}{4} \right\rfloor$	$\phi(n) = (p-1)(q-1)$	$d_2 = u - \phi(n)$
47017	59447	12430	2795019599	2794913864	2794913136	728
56479	43711	12768	2468753569	2468654196	2468653380	816
32801	57737	24936	1893831337	1893744300	1893740800	3500
28499	63313	34814	1804357187	1804272232	1804265376	6856
65519	130973	65454	8581219987	8581034716	8581023496	11220

TABLE 3. Difference of primes versus the distance between $\phi(n)$ and the upper bound u.

Figure 1 below gives a visual representation of the correlation between the difference of primes and the distance between $\phi(n)$ and the upper bound u as observed in Table 3.

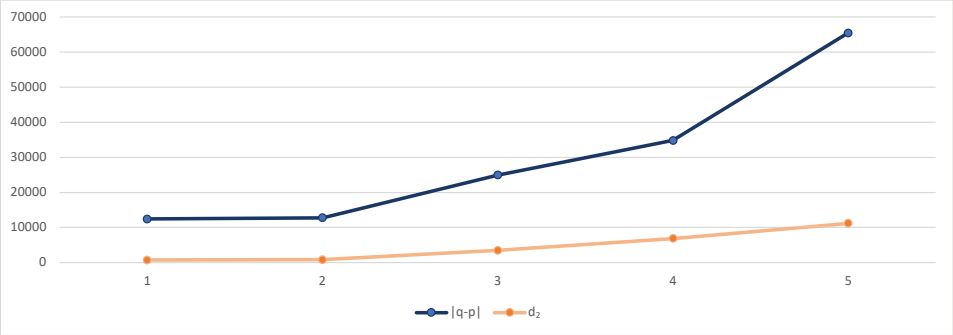

FIGURE 1. Comparison of prime-difference with number of steps required for algorithm

Figure 1 illustrates some proportionality between the number of steps required for our $\phi(n)$-evaluation algorithm and the difference between the two primes. The number of steps seems to gradually increase with an increase in the gap between the two primes. We make the following conjecture.

Conjecture 1. The correlation between the difference of the primes and the distance between $\phi(n)$ and the upper bound u, i.e., $|q-p|$ and d_2 can be given as $d_2 \leq |q-p|^{\epsilon_k}$, where ϵ_k depends on the bit size k of the prime factors. By definition, $\epsilon_k = \max\left\{\frac{\ln d_2}{\ln |q-p|} \mid p \text{ and } q \text{ are } k\text{-bit primes}\right\}$.

To support our above conjecture, we take some examples illustrating the statement of the conjecture. Consider all prime pairs of the same bit size in 12-bit, 13-bit, and 14-bit bit sizes. The quantity ϵ_k given in Table 4 is calculated according to Conjecture 1.

Prime size k	ϵ_k	$\max\{d_2\}$	$\max \{\lvert q-p\rvert\}^{\epsilon_{12}}$	$\max \{\lvert q-p\rvert\}^{\epsilon_{13}}$	$\max \{\lvert q-p\rvert\}^{\epsilon_{14}}$
12	0.767934517	348	348	NA	NA
13	0.787693452	700	594	700	NA
14	0.803839844	1396	NA	1207	1396

TABLE 4. Values of ϵ_k for 12, 13 and 14-bit primes and comparison with $\max\{d_2\}$

The entries NA in Table 4 mean *not applicable* as easily understood. It is apparent from Table 4 that ϵ_k increases with the increase in bit size

k. Furthermore, the difference between the value of $\phi(n)$ and the upper bound u for a prime pair of k bits, which is $|q-p|^{\epsilon_k}$, exceeds $|q-p|^{\epsilon_r}$ for $r < k$. In other words, the value of the exponent of $|q-p|$ for a pair p, q of smaller bit-size is no longer relevant for pairs with larger bit-sizes. We illustrate this correlation through the charts shown below.

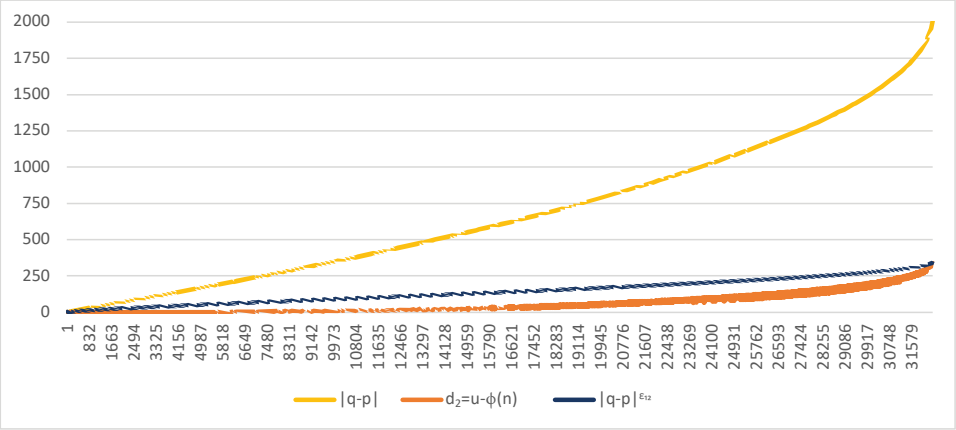

FIGURE 2. For 12-bit prime pairs, $\max\{d_2\} \leq |q-p|^{\epsilon_{12}}$

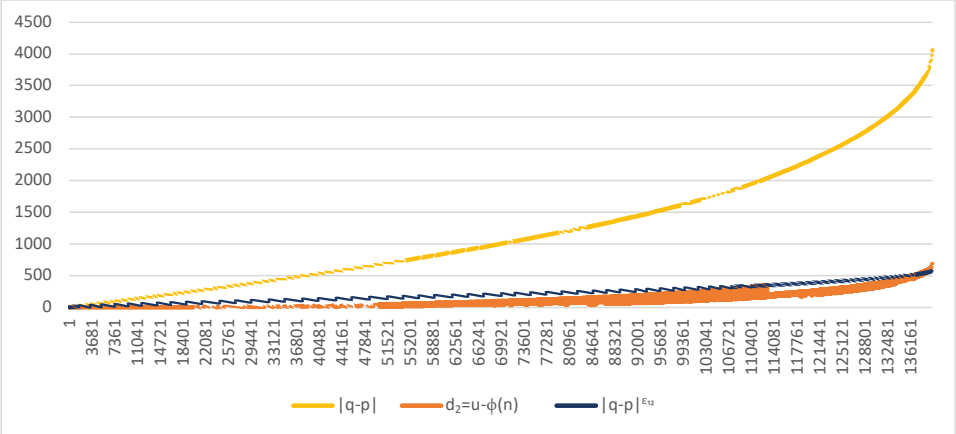

FIGURE 3. For 13-bit prime pairs, $\max\{d_2\} \not\leq |q-p|^{\epsilon_{12}}$

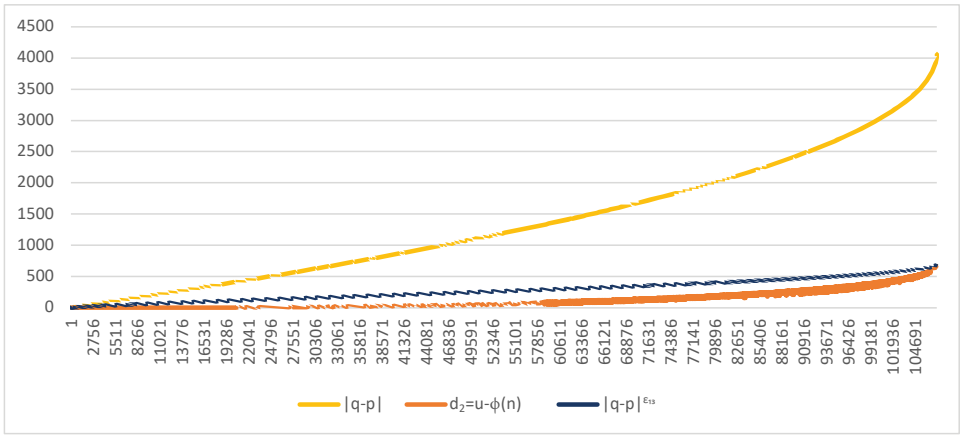

FIGURE 4. For 13-bit prime pairs, $\max\{d_2\} \leq |q-p|^{\epsilon_{13}}$

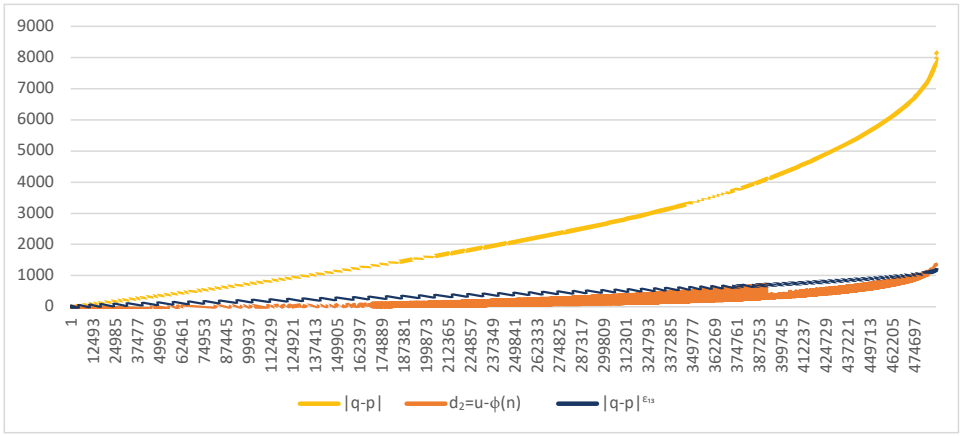

FIGURE 5. For 14-bit prime pairs, $\max\{d_2\} \not\leq |q-p|^{\epsilon_{13}}$

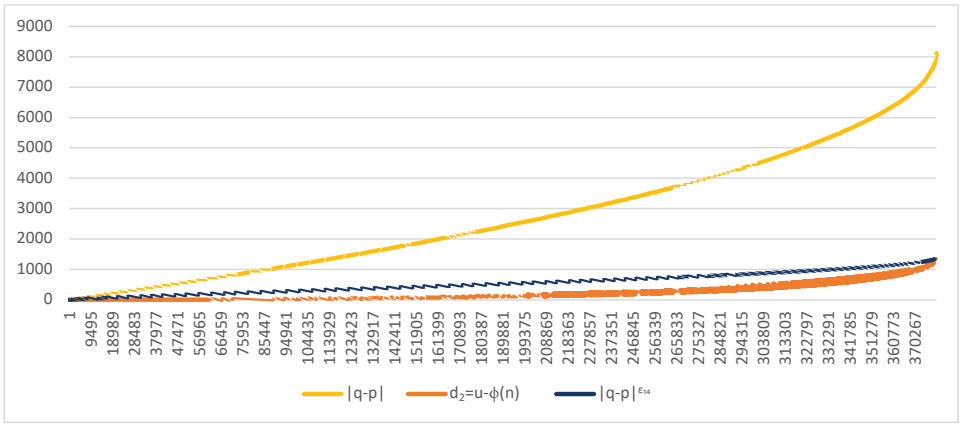

FIGURE 6. For 14-bit prime pairs, $\max\{d_2\} \leq |q-p|^{\epsilon_{14}}$

From Table 4 and Figures 2, 3, 4, 5, 6, we observe that for k-bit prime pairs, and $r < k$, $\max\{|q-p|\}^{\epsilon_r} < \max\{d_2\}$. Hence, the value of ϵ_k continues to increase with the bit size k of the primes. In Table 5, we list the derived values of ϵ_k for primes of k-bits for different values of k to get a better idea of the behavior of ϵ_k with respect to the bit-size k.

Bit-size of primes (k)	ϵ_k
16	0.830399032605173
32	0.917964302323474
64	0.959633228470997
128	0.979975538532857
256	0.990027032916364
512	0.995023274743

TABLE 5. Values of ϵ_k for various prime sizes

From Table 5, we can observe the tendency of ϵ_k to approach 1, i.e., $\epsilon_k \to 1$, as we increase the bit-size k of the prime pairs.

In essence, from all the observations mentioned above, one can infer that this method is best suited when the difference between two prime factors of the RSA modulus is quite small relative to the bit-size of the modulus. It is known that Pollard's $p-1$ method to factorize the RSA modulus, thereby compromising its security, is reliable when either $p-1$ or $q-1$ has smaller prime factors. Likewise, our proposed $\phi(n)$-evaluation algorithm exhibits greater efficiency in breaking the RSA when the prime factors p and q of the RSA modulus n are closer to each other.

5. Conclusion

In order to crack the popular RSA cryptosystem, an adversary requires knowledge of the private decryption key d, which is the modular inverse of the public encryption key e modulo $\phi(n)$. In this paper, we propose an algorithm to expeditiously retrieve the concealed value of $\phi(n)$ only using the public knowledge n. Furthermore, we demonstrate that the time-efficiency of our algorithm is dependent on the distance between two primes and can be highly efficient when the two primes are relatively close as compared to their bit-size. In fact, we conjecture that the difference between the values of $\phi(n)$ and the derived upper bound u is bounded by some exponent of the value of the gap between the two primes, and this exponent increases as the bit-size of the primes increases.

Apart from these, we provide some further scope of research in this direction, such as an analogous algorithm for multi-prime RSA, combining other attacks, and optimizing the lower bound for $\phi(n)$.

6. Future Scope

Refinement in lower bound. In Section 3.3, we determined a very generous lower bound for $\phi(n)$ obtained in (7). This can be further improved, especially for large primes.

Correlation between the time-efficiency and prime gap. We conjectured that the correlation between the difference of primes and the number of steps required to determine the value of $\phi(n)$ can be given as $d_2 = |q - p|^{\epsilon_k}$, where ϵ_k depends on the bit-size k of primes. Further research can be carried out to elucidate this correlation, and a better approximation of ϵ_k can be derived in terms of the bit-size k.

Extension to multi-prime RSA. In multi-prime RSA, the retrieval of $\phi(n)$ does not necessarily give a complete prime factorization of the RSA modulus n. In this scenario, an alternative approach to Algorithm 1 can be investigated to breach the security of multi-prime RSA.

Combination of several different attacks. There are various attacks on RSA as described in [1] and [4]. Certain attacks, when integrated with our proposed algorithm, may substantially enhance the time-efficiency of Algorithm 1.

For instance, the knowledge of some of the least significant bits of $\phi(n)$ can aid in narrowing the range of candidates of $\phi(n)$ resulting in improving the time-efficiency of the algorithm.

7. Statements and Declarations

Competing Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

[1] Dan Boneh. Twenty years of attacks on the RSA cryptosystem. *Notices Amer. Math. Soc.*, 46(2):203–213, 1999.

[2] M Jason Hinek. On the security of multi-prime RSA. *Journal of Mathematical Cryptology*, 2(2):117–147, 2008.

[3] Hendrik W Lenstra Jr. Factoring integers with elliptic curves. *Annals of mathematics*, pages 649–673, 1987.

[4] Majid Mumtaz and Luo Ping. Forty years of attacks on the RSA cryptosystem: A brief survey. *Journal of Discrete Mathematical Sciences and Cryptography*, 22(1):9–29, 2019.

[5] John M Pollard. Theorems on factorization and primality testing. In *Mathematical Proceedings of the Cambridge Philosophical Society*, volume 76(3), pages 521–528. Cambridge University Press, 1974.

[6] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

(Jay Mehta) Department of Mathematics, Sardar Patel University, Vallabh Vidyanagar - 388 120, Gujarat, INDIA.
Email address: jay_mehta@spuvvn.edu

(Hitarth Rana) Department of Mathematics, Sardar Patel University, Vallabh Vidyanagar - 388 120, Gujarat, INDIA.
Email address, Corresponding Author: hitarth.rana@spuvvn.edu